

TP4 — Cache, Artifacts et Variables CI/CD (version debutant)

Objectifs

A la fin de ce TP, vous saurez :

1. La difference entre **cache** et **artifacts** (et quand utiliser chacun)
2. Configurer un cache intelligent base sur `package-lock.json`
3. Utiliser les politiques de cache (`pull`, `push`, `pull-push`)
4. Generer des **rapports de tests JUnit** visibles dans GitLab
5. Generer des **rapports de couverture de code** visibles dans les Merge Requests
6. Utiliser les **variables CI/CD** (predefinies, protegees, masquees)
7. Configurer des **variables dans l'interface GitLab** (Settings)

Prerequis

- Avoir complete le TP3
- Un projet GitLab avec un pipeline fonctionnel
- Savoir faire `git add`, `git commit`, `git push`

Etape 1 — Cache vs Artifacts : comprendre la difference

Avant de coder, il faut comprendre ces deux concepts fondamentaux.

Le cache

Le cache sert a **accelerer les jobs**. Il est telecharge **au debut** du job.

Exemple typique : `node_modules/` (les dependances npm). Au lieu de les reinstaller a chaque pipeline (30 secondes), on les met en cache et on les reutilise (2 secondes).

SANS CACHE :

Pipeline 1 : npm install (30s) + npm test (5s) = 35s

Pipeline 2 : npm install (30s) + npm test (5s) = 35s

Pipeline 3 : npm install (30s) + npm test (5s) = 35s

AVEC CACHE :

Pipeline 1 : npm install (30s) + npm test (5s) = 35s (cree le cache)

Pipeline 2 : cache restore (2s) + npm test (5s) = 7s (5x plus rapide !)

Pipeline 3 : cache restore (2s) + npm test (5s) = 7s

Les artifacts

Les artifacts sont des **fichiers produits par un job** et conserves **apres** le job.

Exemples typiques :

- Le dossier **dist/** (build de l'application) → passe au stage deploy
- Le rapport de tests **junit-report.xml** → affiche dans l'interface GitLab
- Le rapport de couverture → affiche dans les Merge Requests

Tableau comparatif

Aspect	Cache	Artifacts
But	Accelerer les jobs	Conserver les resultats
Quand	Telecharge au debut	Uploade a la fin
Fiabilite	Best-effort (peut etre perdu)	Garanti (stocke sur GitLab)
Telechargeable	Non	Oui (bouton dans l'interface)
Exemple	node_modules/	dist/, coverage/, junit.xml

****Regle simple** :**

- Dependances → ****cache**** (node_modules, .npm, pip cache)
- Resultats → ****artifacts**** (build, rapports, couverture)

Etape 2 — Creer le projet sur GitLab

1. Allez sur <https://gitlab.com>
2. **New project > Create blank project**
3. **Project name** : **tp4-cache-artifacts**
4. **Decochez** "Initialize repository with a README"
5. Cliquez sur **Create project**

Etape 3 — Preparer le code du projet

```
# Aller dans le dossier du jour 2
cd ~/Desktop/"CI CD"/jour2

# Creer un dossier de travail propre
mkdir tp4-work
cd tp4-work

# Copier les fichiers du projet-demo
cp -r ../projet-demo/src ./
cp -r ../projet-demo/tests ./
cp -r ../projet-demo/scripts ./
cp ../projet-demo/package.json ./
cp ../projet-demo/.gitignore ./

# Copier le .gitlab-ci.yml du TP4
cp ../tp4-cache-artifacts/.gitlab-ci.yml ./

# Verifier
ls -la
```

Tester en local :

```
npm install
npm test
```

Vous devez voir **Tests: 17 passed, 17 total**. Puis :

```
rm -rf node_modules
```

Etape 4 — Comprendre le `.gitlab-ci.yml` du TP4

Ouvrez le fichier `.gitlab-ci.yml` dans VS Code :

```
code .gitlab-ci.yml
```

Le pipeline est organise en 4 stages :

Stage	Job(s)	Ce qu'il fait
install	<code>install-deps</code>	Installe les dependances et CREE le cache

test	test-unit, test-integration	Lance les tests et genere les rapports (ARTIFACTS)
build	build-app	Construit l'application (ARTIFACT)
info	pipeline-info	Affiche les variables CI/CD predefinies

Focus : le cache intelligent

```
install-deps:
  cache:
    key:
      files:
        - package-lock.json  # Cle basee sur le CONTENU du fichier
    paths:
      - node_modules/
      - .npm/
    policy: push              # Ce job ECRIT le cache
```

****Pourquoi `key: files: [package-lock.json]` ?****

La cle du cache est calculee a partir du **contenu** de `package-lock.json`. Si vous ne changez pas les dependances, la cle reste la meme et le cache est reutilise. Si vous ajoutez une dependance, la cle change et un nouveau cache est cree.

****Pourquoi `policy: push` sur install et `policy: pull` sur test ?****

- `push` = le job **ecrit** le cache (apres `npm install`)
- `pull` = le job **lit** le cache (pas besoin de reecrire)
- C'est plus rapide que `pull-push` (default) car les jobs de test ne modifient pas les dependances

Focus : les rapports de tests (artifacts reports)

```
test-unit:
  artifacts:
    when: always
    reports:
      junit: junit-report.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
```

****Que fait `reports: junit` ?****

GitLab lit le fichier `junit-report.xml` et affiche les resultats des tests ****directement dans l'interface**** :

- Dans la page du pipeline : nombre de tests passes/echoues
- Dans les Merge Requests : liste detaillee de chaque test

****Que fait `reports: coverage\_report` ?****

GitLab lit le rapport Cobertura et affiche la ****couverture de code ligne par ligne**** dans les Merge Requests. Vous verrez quelles lignes sont testees (vert) et lesquelles ne le sont pas (rouge).

Etape 5 — Pousser vers GitLab

```
git init -b main
git add .
git status
git commit -m "init: TP4 cache artifacts et variables"
git remote add origin https://gitlab.com/votre-user/tp4-cache-artifacts.git
git push -u origin main
```

Etape 6 — Observer le pipeline et les rapports

6.1 — Le pipeline

1. Allez sur GitLab > **Build** > **Pipelines**
2. Cliquez sur le pipeline
3. Observez les **4 stages** et les 5 jobs

6.2 — Les rapports de tests JUnit

4. Cliquez sur le job **test-unit**
5. En haut de la page du job, vous devez voir un onglet **Tests** avec le nombre de tests passes
6. Cliquez dessus pour voir le detail de chaque test (nom, duree, statut)

6.3 — La couverture de code

7. Retournez sur la page du pipeline
8. Vous devez voir un pourcentage de couverture a cote du job **test-unit** (ex: **75%**)

6.4 — Telecharger les artifacts

9. Sur la page du pipeline, cliquez sur le bouton de téléchargement (icone fleche vers le bas) a droite
10. Vous pouvez telecharger les artifacts de chaque job (dist/, coverage/, etc.)

Etape 7 — Observer le cache en action

Premier pipeline (cache froid)

Dans les logs du job `install-deps`, vous verrez :

```
Restoring cache...  
WARNING: cache not found <-- Normal au premier run  
...  
Saving cache for successful job...  
Creating cache node_modules-abc123...
```

Deuxieme pipeline (cache chaud)

Faites une petite modification et poussez :

```
echo "// commentaire" >> src/utils.js  
git add .  
git commit -m "test: verifier le cache"  
git push
```

Dans les logs du nouveau `install-deps`, vous verrez :

```
Restoring cache...  
Successfully extracted cache <-- Le cache est reutilise !
```

Et le job sera **beaucoup plus rapide** que la premiere fois.

****C'est le but du cache**** : apres le premier pipeline, les suivants sont 3 a 5 fois plus rapides car on ne reinstalle pas les dependances.

Etape 8 — Comprendre les variables CI/CD

Variables predefinies

Le job `pipeline-info` affiche les variables predefinies par GitLab. Cliquez sur ce job dans le pipeline pour voir :

- `CI_COMMIT_SHORT_SHA` : le hash court du commit (ex: `a1b2c3d`)

- **CI_COMMIT_BRANCH** : la branche (ex: **main**)
- **CI_PIPELINE_ID** : l'identifiant unique du pipeline
- **CI_PROJECT_NAME** : le nom du projet
- **CI_JOB_NAME** : le nom du job en cours
- etc.

****A quoi servent ces variables ?****

Elles permettent de personnaliser le comportement du pipeline. Par exemple :

- Deployer uniquement sur la branche `main` : `if: '$CI_COMMIT_BRANCH == "main"'``
- Nommer les artifacts avec le hash du commit : `name: "build-${CI_COMMIT_SHORT_SHA}"``
- Taguer une image Docker : `docker build -t mon-app:${CI_COMMIT_SHORT_SHA} .``

Variables personnalisées (dans l'interface GitLab)

Vous pouvez créer vos propres variables dans GitLab :

1. Allez dans votre projet > **Settings** > **CI/CD**
2. Développez la section **Variables**
3. Cliquez **Add variable**
4. Remplissez :
 - **Key** : **DEPLOY_TOKEN** (le nom de la variable)
 - **Value** : **mon-secret-123** (la valeur)
 - **Flags** :
 - **Protected** : la variable n'est disponible que sur les branches protégées (**main**)
 - **Masked** : la valeur est masquée dans les logs (**[MASKED]** au lieu de **mon-secret-123**)
5. Cliquez **Add variable**

****Quand utiliser les variables GitLab ?****

Pour tout ce qui est ****secret**** ou ****configurable**** :

- Mots de passe de base de données
- Tokens d'API
- URLs de serveurs de déploiement
- Configuration par environnement (staging vs production)

****REGLE ABSOLUE**** : ne JAMAIS mettre de secrets dans le `.gitlab-ci.yml` ou dans le code. Utilisez toujours les variables GitLab protégées et masquées.

Etape 9 — Exercice pratique : créer une variable et l'utiliser

9.1 — Créer la variable dans GitLab

1. Settings > CI/CD > Variables > Add variable
2. Key : `WELCOME_MESSAGE`
3. Value : `Bonjour depuis la variable GitLab !`
4. Laissez les flags decoches (pas protegee, pas masquee pour cet exercice)
5. Add variable

9.2 — Modifier le pipeline pour utiliser la variable

Ajoutez ce job a la fin de votre `.gitlab-ci.yml` :

```
test-variable:
  stage: info
  image: node:18-alpine
  script:
    - echo "Le message est :"
```

9.3 — Pousser et observer

```
git add .gitlab-ci.yml
git commit -m "feat: ajouter job qui utilise une variable GitLab"
git push
```

Dans les logs du job `test-variable`, vous devez voir :

```
Le message est :
Bonjour depuis la variable GitLab !
```

La variable definie dans l'interface GitLab est bien injectee dans le job.

Recapitulatif du TP4

Concept	Ce que vous avez appris
Cache	Accelere les jobs en sauvegardant les dependances (<code>node_modules/</code>)
Cle de cache	Basee sur <code>package-lock.json</code> : change uniquement si les dependances changent
Policy pull/push	<code>push</code> = ecrit le cache, <code>pull</code> = lit le cache
Artifacts	Fichiers produits par un job et conserves (build, rapports)
JUnit report	Affiche les resultats des tests dans l'interface GitLab
Cobertura report	Affiche la couverture de code dans les Merge Requests
Variables predefinies	Informations automatiques (commit, branche, pipeline, runner)

Variables GitLab	Valeurs configurees dans Settings > CI/CD > Variables
Protected / Masked	Securise les secrets (branche protegee + masquage dans les logs)

Erreurs courantes et solutions (lecons du jour 1)

Erreur	Cause	Solution
<code>yaml invalid</code> + script config error	<code>:</code> dans une ligne de script	Entourer la ligne de guillemets simples <code>'...'</code>
<code>Updates were rejected</code> au push	Depot GitLab initialise avec README	Decoher README a la creation
<code>npm ci can only install with existing package-lock</code>	Pas de <code>package-lock.json</code>	Faire <code>npm install</code> en local une fois pour le generer
Cache jamais reutilise	Cle qui change a chaque commit	Utiliser <code>key: files: [package-lock.json]</code> (stable)
Artifacts non visibles dans GitLab	<code>reports: junit</code> mal configure	Verifier que le chemin vers le fichier XML est correct

Fin du TP4. Vous maitrisez maintenant les concepts fondamentaux d'optimisation de pipeline CI/CD.